Strategy

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

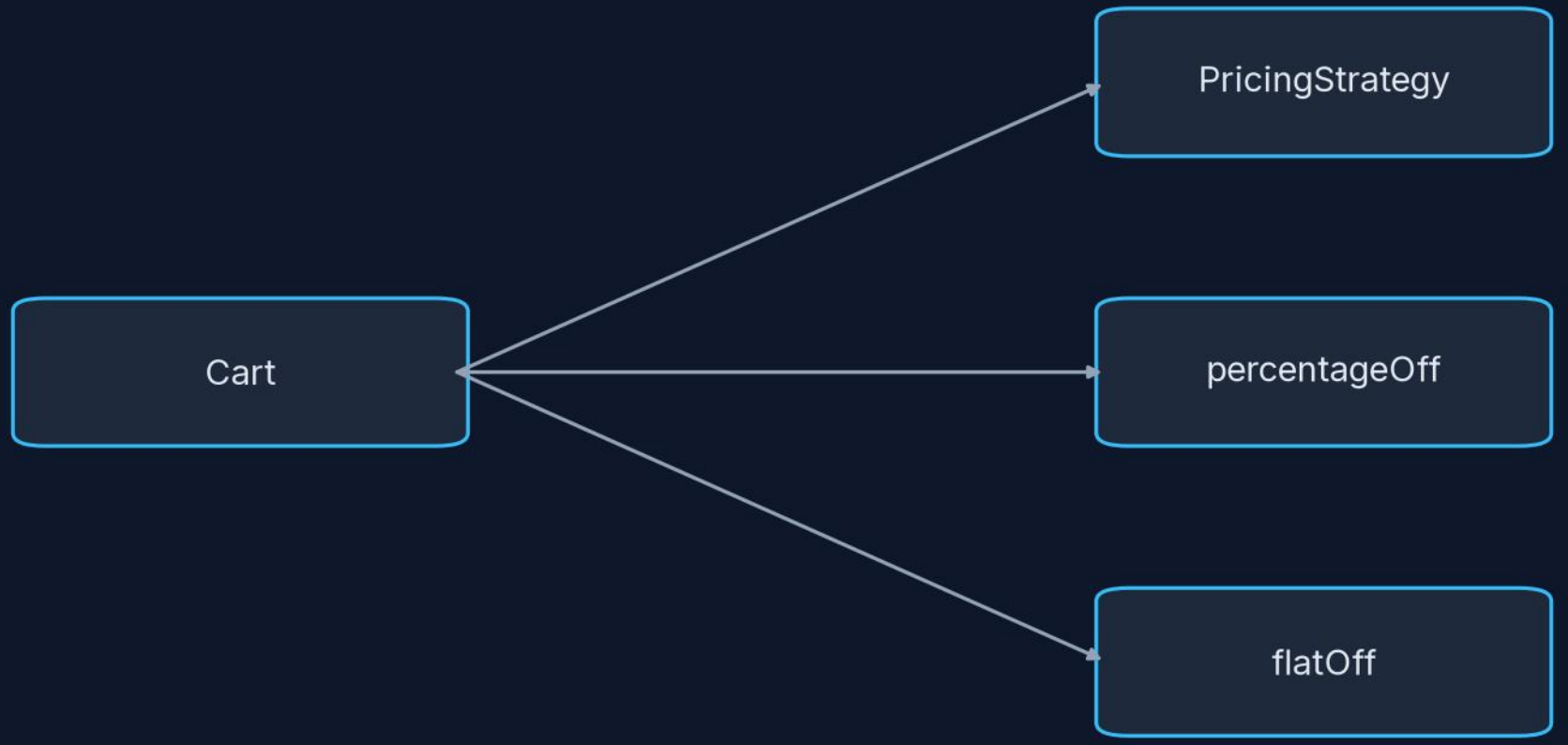
Today's Hook

- Today: Strategy as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Strategy structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Strategy
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/strategy/Cart.java
package com.buildwithwaqar.patterns.gof.behavioral.strategy;

import java.util.function.IntUnaryOperator;

interface PricingStrategy {
    int priceCents(int baseCents);

    static PricingStrategy percentageOff(int percent) {
        return base -> base - base * percent / 100;
    }

    static PricingStrategy flatOff(int cents) {
        return base -> Math.max(0, base - cents);
    }
}

/** Context that delegates to a pluggable pricing strategy. */
public final class Cart {
    private final PricingStrategy strategy;

    public Cart(PricingStrategy strategy) { this.strategy = strategy; }
```

Implementation (2)

```
    /** Java 25 style: accept a function directly. */
    public Cart(IntUnaryOperator fn) { this.strategy = fn::applyAsInt; }

    public int checkout(int baseCents) { return strategy.priceCents(baseCents); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.gof.behavioral.strategy;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Strategy walkthrough ===");
    System.out.println("pct10=" + new Cart(PricingStrategy.percentageOff(10)).checkout(1000));
    System.out.println("flat200=" + new Cart(PricingStrategy.flatOff(200)).checkout(1000));
    PricingStrategy half = base -> base / 2;
    System.out.println("half=" + new Cart(half).checkout(1000));
  }
}
```

Walkthrough Output

```
=== Strategy walkthrough ===
pct10=900
flat200=800
half=500
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Strategy
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Strategy → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.